Project Title

StockSense AI — Smart Inventory, Sales Analytics & Demand Forecasting for Small Businesses

Problem Statement

Small shops, local brands, and small D2C sellers usually track sales in messy Excel sheets or not at all. Because of that, they overstock slow-moving items, run out of fast-moving ones, and make decisions by guesswork instead of data.

This project solves that by turning raw sales data into:

- **clear dashboards,**
- **demand forecasts,**
- **reorder suggestions,**
- **and business insights.**

StockSense AI — What Exactly Each Developer Will Build

Think of it as **2 separate modules**:

Developer 1 (Full-Stack)

Builds the **product that users see and interact with**.

Developer 2 (Python/Data Science)

Builds the **brain that analyzes data and generates insights**.

Final User Flow

A shop owner:

1. Creates account
2. Adds products
3. Uploads sales CSV
4. Clicks "Analyze"

5. Sees dashboard
6. Gets insights:
    - Best-selling products
    - Slow-moving products
    - Revenue trends
    - Forecasted demand
    - Reorder suggestions

Developer 1 (Full-Stack) Responsibilities

Frontend Pages

1. Login / Signup Page

Fields:

- Name
- Email
- Password

---

2. Dashboard Page

Shows:

Cards:

- Total Revenue
- Total Products
- Total Sales
- Low Stock Products

Charts:

- Sales Trend
- Revenue Trend

Tables:

- Top Selling Products
- Low Performing Products

---

3. Product Management Page

User can:

- Add Product
- Edit Product
- Delete Product

Fields:

- Product Name
- Category
- Price
- Current Stock
- Reorder Level

---

4. Sales Upload Page

Upload:

Date,Product,Quantity,Revenue

2026-01-01,Notebook,15,750

2026-01-01,Pen,20,200

Store data in database.

---

5. AI Insights Page

Shows output coming from Python API.

Example:

Notebook demand increasing

Expected stockout in 5 days

Pen sales dropped 22%

Recommended reorder quantity: 100

Backend APIs

Developer 1 creates:

Auth APIs

POST /register

POST /login

Product APIs

GET /products

POST /products

PUT /products

DELETE /products

Sales APIs

POST /upload-sales

GET /sales

AI Integration API

GET /analytics

This calls Developer 2's service.

Developer 2 (Python/Data Science) Responsibilities

This person never worries about UI.

Only data and analytics.

Module 1: Data Cleaning

Input:

Date,Product,Quantity,Revenue

Tasks:

- Remove duplicates
- Fix missing values
- Convert dates
- Validate records

Libraries:

Pandas

NumPy

Module 2: Sales Analytics

Generate:

Top Selling Products

Example:

Notebook

Laptop Bag

Mouse

Slow Moving Products

Example:

Marker

Stapler

Tape

Revenue Analysis

Calculate:

- Total revenue
- Weekly revenue
- Monthly revenue

Module 3: Demand Forecasting

Predict:

How many units

will sell next week?

Example:

Current sales:

10

12

14

15

18

Prediction:

22 units next week

Libraries:

Scikit-learn

Statsmodels

Prophet (optional)

Module 4: Reorder Recommendation

Logic:

Current Stock:

40

Predicted Demand:

100

Output:

Reorder 60 units

Module 5: Insight Generator

Convert numbers into readable insights.

Example:

Instead of:

Revenue +25%

Show:

Revenue increased by 25%

compared to last week.

Module 6: FastAPI Service

Developer 2 creates APIs:

GET /forecast

GET /top-products

GET /slow-products

GET /insights

Returns JSON.

Example:

```
{
  "forecast": 120,
  "insight": "Demand increasing"
}
```

Communication Between Both Developers

Frontend

↓

Backend

↓

Python Analytics Service

↓

Database

Example:

User clicks:

Analyze Sales

Backend sends request:

GET /forecast

Python service responds:

```
{
  "Notebook": 150
}
```

Frontend displays:

Notebook Forecast:

150 Units

Tech Stack

Full Stack Developer

Frontend

- React.js
- Tailwind CSS
- Chart.js

Skills shown:

- React
- API integration
- UI design

Backend

- Node.js
- Express.js

Skills shown:

- REST APIs
- Authentication
- CRUD

Database

- PostgreSQL

Skills shown:

- Database design
- SQL

Python Developer

Data Processing

- Pandas
- NumPy

Analysis

- Pandas
- Matplotlib

Forecasting

- Scikit-learn
- Statsmodels

API Service

- FastAPI

Folder Structure

project/

frontend/

|

├— Dashboard

├— Products

├— Upload

├— Insights

backend/

|

├— Auth

├— Product APIs

├— Sales APIs

├— Database

analytics/

|

├— data_cleaning.py

├— forecasting.py

├— insights.py

├── recommendation.py

├── main.py

What You Can Complete in 7 Days

Full-Stack Student

✅ Login

✅ Product CRUD

✅ Sales Upload

✅ Dashboard

✅ Charts

✅ Deploy

Python Student

✅ CSV Processing

✅ Analytics

✅ Forecasting

✅ Insights

✅ FastAPI

At the end of the week you'll have a deployed web app where a user uploads sales data and instantly gets business analytics and AI-based inventory recommendations—a project that looks much closer to a real SaaS product than a typical student CRUD application.